```
import os, random, zipfile
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay
tf.random.set_seed(42)
np.random.seed(42)

#https://drive.google.com/file/d/14nSh4QwFKsBoH9OjudLhjcFj7O02WS6k/
view?usp=drive_link

FILE_ID="14nSh4QwFKsBoH9OjudLhjcFj7O02WS6k"
ARCHIVE="/content/animals.zip"

!gdown {FILE_ID} -O {ARCHIVE}
print("downloaded: {os.path.getsize(ARCHIVE)/1e6:.1f} MB")
```

```
Downloading...
From (original): https://drive.google.com/uc?
id=14nSh4QwFKsBoH9OjudLhjcFj7O02WS6k
From (redirected): https://drive.google.com/uc?
id=14nSh4QwFKsBoH9OjudLhjcFj7O02WS6k&confirm=t&uuid=182e4e93-536c-
49f6-bd8e-0fcd43faee11
To: /content/animals.zip
100% 68.6M/68.6M [00:01<00:00, 48.7MB/s]
downloaded: {os.path.getsize(ARCHIVE)/1e6:.1f} MB
```

```
DATA_DIR="/content/data"
with zipfile.ZipFile(ARCHIVE) as z:
  z.extractall(DATA_DIR,members=[m for m in z.namelist() if not
m.startswith(("_MACOSX"))])

print(os.listdir(DATA_DIR))
```

```
['cats_and_dogs_filtered']
```

```
for root, dirs, files in os.walk(DATA_DIR):
    print(root, "->", len(files), "files")
```

```
/content/data -> 0 files
/content/data/cats_and_dogs_filtered -> 1 files
/content/data/cats_and_dogs_filtered/validation -> 0 files
/content/data/cats_and_dogs_filtered/validation/dogs -> 500 files
/content/data/cats_and_dogs_filtered/validation/cats -> 500 files
```

```
/content/data/cats_and_dogs_filtered/train -> 0 files
/content/data/cats_and_dogs_filtered/train/dogs -> 1000 files
/content/data/cats_and_dogs_filtered/train/cats -> 1000 files
```

```
TRAIN_DIR="/content/data/cats_and_dogs_filtered/train"
TRAIN_CAT="/content/data/cats_and_dogs_filtered/train/cats"
TRAIN_DOG="/content/data/cats_and_dogs_filtered/train/dogs"

n_cats=len(os.listdir(TRAIN_CAT))
n_dogs=len(os.listdir(TRAIN_DOG))

print(n_cats)
print(n_dogs)
```

```
1000
1000
```

```
print(os.listdir(TRAIN_DIR))
```

```
['dogs', 'cats']
```

```
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
train_ds=keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
    validation_split=0.2,
    subset="training",
    seed=42,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
val_ds=keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
    validation_split=0.2,
    subset="validation",
    seed=42,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
```

```
Found 2000 files belonging to 2 classes.
Using 1600 files for training.
Found 2000 files belonging to 2 classes.
Using 400 files for validation.
```

```
class_names = train_ds.class_names
num_classes = len(class_names)

print("Classes:", class_names)
print("Number of classes:", num_classes)
```

```
Classes: ['cats', 'dogs']
Number of classes: 2

plt.figure(figsize=(10, 10))

for images, labels in train_ds.take(1):
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")

plt.show()
```




```
data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1)
])

base_model = keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet"
)
```

```
base_model.trainable = False

print("Pretrained model loaded!")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/mobilenet_v2/
mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Pretrained model loaded!
```

```
inputs = keras.Input(shape=(224, 224, 3))

# Data augmentation
x = data_augmentation(inputs)

# MobileNetV2 preprocessing
x = keras.applications.mobilenet_v2.preprocess_input(x)

# Extract features using pretrained model
x = base_model(x, training=False)

# Classification head
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)

outputs = layers.Dense(
    num_classes,
    activation="softmax"
)(x)

model = keras.Model(inputs, outputs)

model.summary()
```

```
Model: "functional_2"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
━━━┩
│ input_layer_2 (InputLayer)           │ (None, 224, 224, 3)         │
0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
──┤
│ sequential_1 (Sequential)            │ (None, 224, 224, 3)         │
0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
──┤
```

```
| true_divide (TrueDivide)         | (None, 224, 224, 3)     |
0 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| subtract (Subtract)              | (None, 224, 224, 3)     |
0 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| mobilenetv2_1.00_224             | (None, 7, 7, 1280)      |
2,257,984 |
| (Functional)                     |                         |
|
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| global_average_pooling2d         | (None, 1280)            |
0 |
| (GlobalAveragePooling2D)         |                         |
|
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| dropout (Dropout)                | (None, 1280)            |
0 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| dense_1 (Dense)                  | (None, 2)               |
2,562 |
└──────────────────────────────────┴─────────────────────────┴──────────
──────┘

 Total params: 2,260,546 (8.62 MB)

 Trainable params: 2,562 (10.01 KB)

 Non-trainable params: 2,257,984 (8.61 MB)
```

```python
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10
)
```

```
Epoch 1/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 102s 2s/step - accuracy: 0.8400 - loss:
0.3484 - val_accuracy: 0.9775 - val_loss: 0.0900
Epoch 2/10
```

```
50/50 ━━━━━━━━━━━━━━━━━━━━ 142s 2s/step - accuracy: 0.9588 - loss:
0.1115 - val_accuracy: 0.9875 - val_loss: 0.0695
Epoch 3/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 136s 2s/step - accuracy: 0.9675 - loss:
0.0870 - val_accuracy: 0.9875 - val_loss: 0.0610
Epoch 4/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 147s 2s/step - accuracy: 0.9663 - loss:
0.0809 - val_accuracy: 0.9875 - val_loss: 0.0596
Epoch 5/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 95s 2s/step - accuracy: 0.9719 - loss:
0.0805 - val_accuracy: 0.9850 - val_loss: 0.0664
Epoch 6/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 142s 2s/step - accuracy: 0.9762 - loss:
0.0705 - val_accuracy: 0.9900 - val_loss: 0.0555
Epoch 7/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 96s 2s/step - accuracy: 0.9812 - loss:
0.0607 - val_accuracy: 0.9900 - val_loss: 0.0546
Epoch 8/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 97s 2s/step - accuracy: 0.9737 - loss:
0.0624 - val_accuracy: 0.9875 - val_loss: 0.0512
Epoch 9/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 141s 2s/step - accuracy: 0.9769 - loss:
0.0551 - val_accuracy: 0.9900 - val_loss: 0.0514
Epoch 10/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 142s 2s/step - accuracy: 0.9750 - loss:
0.0566 - val_accuracy: 0.9875 - val_loss: 0.0566
```

```
val_loss, val_acc = model.evaluate(val_ds)

print("Validation Loss:", val_loss)
print("Validation Accuracy:", val_acc * 100, "%")
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 15s 1s/step - accuracy: 0.9875 - loss:
0.0566
Validation Loss: 0.056626126170158386
Validation Accuracy: 98.7500011920929 %
```

```
plt.figure(figsize=(12, 4))

# Accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

# Loss
plt.subplot(1, 2, 2)
```

```
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.show()
```



